

Apigee X Simulator

Policy Validation Test Results

Generated: 2026-07-17T03:20:41.315Z

These results were produced by directly invoking the real, unmodified policy executors from **backend/src/utis/policyExecutors.ts** (the same code that runs during Trace), with hand-built request contexts and, for OAuthV2, an in-memory stand-in for the Prisma database (this sandbox environment could not reach the Prisma engine download host, so a live SQLite-backed run wasn't possible here - the harness that produced this data is not part of the delivered project).

Total Tests	Passed	Failed
42	42	0

Results by Policy / Scenario Group

Policy / Group	Tests	Passed	Failed
VerifyAPIKey	3	3	0
OAuthV2	7	7	0
Header Case-Insensitivity Fix	1	1	0
VerifyJWT	5	5	0
BasicAuthentication	3	3	0
CORS	1	1	0
AccessControl	2	2	0
SpikeArrest	1	1	0
Quota	1	1	0
ResponseCache / PopulateCache / LookupCache	1	1	0
AssignMessage	1	1	0
ExtractVariables	1	1	0
RaiseFault	1	1	0
JSONToXML	1	1	0
XMLToJSON	1	1	0
JavaScript	1	1	0
Route Rules (multi-method target routing)	4	4	0

Policy / Group	Tests	Passed	Failed
Connector Stubs (simulated)	7	7	0

VerifyAPIKey

1. Valid API key (header) -> success

PASS

Policy type: **VerifyAPIKey**

Config:

```
{
  "apiKeyLocation": "header",
  "apiKeyParam": "x-api-key"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {
    "x-api-key": "test-client-id"
  },
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "success",
  "message": "API key verified for app \"Test App\"",
  "variablesSet": {
    "verifyapikey.client_id": "test-client-id",
    "verifyapikey.developer.app.name": "Test App",
    "verifyapikey.developer.email": "dev@example.com",
    "verifyapikey.apiproduct.names": ""
  },
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

2. Missing API key -> 401

PASS

Policy type: **VerifyAPIKey**

Config:

```
{
  "apiKeyLocation": "header",
  "apiKeyParam": "x-api-key"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {},
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "error",
  "message": "API key not found in request",
  "variablesSet": null,
  "faultResponse": {
    "status": 401,
    "reasonPhrase": "Unauthorized",
    "body": {
      "fault": {
        "faultstring": "Failed to resolve API Key variable request.x-api-key",
        "detail": {
          "errorcode": "steps.oauth.v2.FailedToResolveAPIKey"
        }
      }
    }
  }
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

3. Invalid API key -> 401

PASS

Policy type: **VerifyAPIKey**

Config:

```
{
  "apiKeyLocation": "header",
  "apiKeyParam": "x-api-key"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {
    "x-api-key": "not-a-real-key"
  },
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "error",
  "message": "Invalid API key: not-a-real-key",
  "variablesSet": null,
  "faultResponse": {
    "status": 401,
    "reasonPhrase": "Unauthorized",
    "body": {
      "fault": {
        "faultstring": "Invalid API Key",
        "detail": {
          "errorcode": "oauth.v2.InvalidApiKey"
        }
      }
    }
  }
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

OAuthV2

4. GenerateAccessToken with valid client credentials -> success

PASS

Policy type: OAuthV2

Config:

```
{
  "operation": "GenerateAccessToken",
  "grantType": "client_credentials",
  "scopes": "read"
}
```

Request (as passed to the executor):

```
{
  "method": "POST",
  "path": "/oauth/token",
  "headers": {},
  "query": {},
  "body": {
    "client_id": "test-client-id",
    "client_secret": "test-client-secret"
  }
}
```

Result:

```
{
  "status": "success",
  "message": "Access token issued for app \"Test App\" (grant_type=client_credentials)",
  "variablesSet": {
    "oauthv2.access_token": "sim_at_u43fgwso4oqlrko6",
    "oauthv2.client_id": "test-client-id",
    "oauthv2.developer.app.name": "Test App"
  },
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {
    "content-type": "application/json"
  },
  "body": {
    "access_token": "sim_at_u43fgwso4oqlrko6",
    "token_type": "Bearer",
    "expires_in": 3600,
    "scope": "read"
  }
}
```

5. GenerateAccessToken response contains access_token

PASS

Policy type: OAuthV2

Config:

```
{
  "operation": "GenerateAccessToken"
}
```

Request (as passed to the executor):

```
{
  "method": "POST",
  "path": "/oauth/token",
  "headers": {},
  "query": {},
  "body": {
    "client_id": "test-client-id",
    "client_secret": "test-client-secret"
  }
}
```

Result:

```
{
  "status": "success",
  "message": "Access token issued for app \"Test App\" (grant_type=client_credentials)",
  "variablesSet": {
    "oauthv2.access_token": "sim_at_d0gpaunycxoixgeq",
    "oauthv2.client_id": "test-client-id",
    "oauthv2.developer.app.name": "Test App"
  },
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {
    "content-type": "application/json"
  },
  "body": {
    "access_token": "sim_at_d0gpaunycxoixgeq",
    "token_type": "Bearer",
    "expires_in": 3600,
    "scope": ""
  }
}
```

6. GenerateAccessToken with wrong secret -> 401

PASS

Policy type: OAuthV2

Config:

```
{  
  "operation": "GenerateAccessToken"  
}
```

Request (as passed to the executor):

```
{  
  "method": "POST",  
  "path": "/oauth/token",  
  "headers": {},  
  "query": {},  
  "body": {  
    "client_id": "test-client-id",  
    "client_secret": "WRONG"  
  }  
}
```

Result:

```
{  
  "status": "error",  
  "message": "Invalid client credentials for client_id test-client-id",  
  "variablesSet": null,  
  "faultResponse": {  
    "status": 401,  
    "reasonPhrase": "Unauthorized",  
    "body": {  
      "error": "invalid_client",  
      "error_description": "Client authentication failed"  
    }  
  }  
}
```

Response / context after execution:

```
{  
  "status": 200,  
  "reasonPhrase": "OK",  
  "headers": {},  
  "body": null  
}
```


7. VerifyAccessToken with just-issued token (Header location) -> success

PASS

Policy type: OAuthV2

Config:

```
{
  "operation": "VerifyAccessToken",
  "accessTokenLocation": "Header",
  "accessTokenHeaderName": "Authorization"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {
    "authorization": "Bearer sim_at_u43fgwso4oqlrko6"
  },
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "success",
  "message": "Access token verified (client_id test-client-id)",
  "variablesSet": {
    "oauth2.client_id": "test-client-id",
    "oauth2.scope": "read"
  },
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

8. VerifyAccessToken with token in query param (new feature) -> success

PASS

Policy type: OAuthV2

Config:

```
{
  "operation": "VerifyAccessToken",
  "accessTokenLocation": "Query Parameter",
  "accessTokenQueryParam": "access_token"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/secure",
  "headers": {},
  "query": {
    "access_token": "sim_at_u43fgwso4oqlrko6"
  },
  "body": null
}
```

Result:

```
{
  "status": "success",
  "message": "Access token verified (client_id test-client-id)",
  "variablesSet": {
    "oauth2.client_id": "test-client-id",
    "oauth2.scope": "read"
  },
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

9. VerifyAccessToken with no token -> 401 missing

PASS

Policy type: OAuthV2

Config:

```
{
  "operation": "VerifyAccessToken",
  "accessTokenLocation": "Header",
  "accessTokenHeaderName": "Authorization"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {},
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "error",
  "message": "Missing access token - expected it in the header \"Authorization\"",
  "variablesSet": null,
  "faultResponse": {
    "status": 401,
    "reasonPhrase": "Unauthorized",
    "body": {
      "fault": {
        "faultstring": "Invalid Access Token",
        "detail": {
          "errorcode": "keymanagement.service.invalid_access_token"
        }
      }
    }
  }
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

10. VerifyAccessToken with bogus token -> 401 invalid

PASS

Policy type: OAuthV2

Config:

```
{
  "operation": "VerifyAccessToken"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {
    "authorization": "Bearer not-a-real-token"
  },
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "error",
  "message": "Access token is invalid or was never issued by this simulator",
  "variablesSet": null,
  "faultResponse": {
    "status": 401,
    "reasonPhrase": "Unauthorized",
    "body": {
      "fault": {
        "faultstring": "Invalid Access Token",
        "detail": {
          "errorcode": "keymanagement.service.invalid_access_token"
        }
      }
    }
  }
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

Header Case-Insensitivity Fix

11. "Authorization" (capital, as typed in Trace UI) normalizes to lowercase "authorization" key

PASS

Policy type: traceEngine.ts ingestion

Request (as passed to the executor):

```
{
  "rawHeaders": {
    "Authorization": "Bearer some-token-value"
  }
}
```

Result:

```
{
  "status": "success",
  "message": "normalized = {\"authorization\": \"Bearer some-token-value\"}",
  "variablesSet": null,
  "faultResponse": null
}
```

VerifyJWT

12. Valid JWT (issuer/audience/algorithm/expiry all match) -> success

PASS

Policy type: **VerifyJWT**

Config:

```
{
  "algorithm": "RS256",
  "issuer": "https://accounts.example.com",
  "audience": "my-api"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {
    "authorization": "Bearer eyJhbGciOiJSUzI1NiJ9.eyJpc3MiOiJodHRwczovL2FjY291bnRzLmV4YW1wbGUuY29tIiwiaXVkiJjoibXktYXBpIiwiaWZlbnR1bSI6ImF1dG8iLCJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9"
  },
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "success",
  "message": "JWT verified (algorithm, expiry, issuer, and audience all checked)",
  "variablesSet": {
    "jwt.decoded.header": {
      "alg": "RS256"
    },
    "jwt.decoded.claims": {
      "iss": "https://accounts.example.com",
      "aud": "my-api",
      "exp": 1784262041
    }
  },
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

PASS

Config:

Request (as passed to the executor):

Result:

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

14. Wrong issuer -> 401

PASS

Policy type: **VerifyJWT**

Config:

```
{
  "issuer": "https://accounts.example.com",
  "audience": "my-api"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {
    "authorization": "Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJodHRwczovL2V2aWwuZXhhbXBsZS5jb20iLCJhdwQiOiJte
  },
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "error",
  "message": "JWT issuer mismatch: token issuer is \"https://evil.example.com\", policy expects \"https://accounts.example.com\"",
  "variablesSet": null,
  "faultResponse": {
    "status": 401,
    "reasonPhrase": "Unauthorized",
    "body": {
      "fault": {
        "faultstring": "JWT issuer is invalid",
        "detail": {
          "errorcode": "steps.jwt.InvalidIssuer"
        }
      }
    }
  }
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

PASS

Policy type: **VerifyJWT**

Config:

```
{
  "algorithm": "RS256",
  "issuer": "https://accounts.example.com",
  "audience": "my-api"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {
    "authorization": "Bearer eyJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJodHRwczovL2FjY291bnRzLmV4YW1wbGUuY29tIiwiaXVkIjoibXktYXBpIiwiaWF0IjoiMTkxMTEyOTQifQ."
  },
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "error",
  "message": "JWT algorithm mismatch: token uses \"HS256\", policy expects \"RS256\"",
  "variablesSet": null,
  "faultResponse": {
    "status": 401,
    "reasonPhrase": "Unauthorized",
    "body": {
      "fault": {
        "faultstring": "JWT algorithm mismatch",
        "detail": {
          "errorcode": "steps.jwt.AlgorithmMismatch"
        }
      }
    }
  }
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```


16. Malformed JWT -> 401

PASS

Policy type: **VerifyJWT**

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {
    "authorization": "Bearer not.a.validjwt.at.all"
  },
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "error",
  "message": "JWT is malformed (expected 3 segments)",
  "variablesSet": null,
  "faultResponse": {
    "status": 401,
    "reasonPhrase": "Unauthorized",
    "body": {
      "fault": {
        "faultstring": "JWT is not valid",
        "detail": {
          "errorcode": "steps.jwt.InvalidToken"
        }
      }
    }
  }
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

BasicAuthentication

17. Decode valid Basic header -> success

PASS

Policy type: **BasicAuthentication**

Config:

```
{
  "operation": "Decode",
  "user": "req.user",
  "password": "req.pass"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {
    "authorization": "Basic YWxpY2U6czNjcmV0"
  },
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "success",
  "message": "Basic auth decoded",
  "variablesSet": {
    "req.user": "alice",
    "req.pass": "*****"
  },
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

18. Decode with no Basic header -> 401

PASS

Policy type: **BasicAuthentication**

Config:

```
{
  "operation": "Decode",
  "user": "req.user",
  "password": "req.pass"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {},
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "error",
  "message": "Missing Basic Authorization header",
  "variablesSet": null,
  "faultResponse": {
    "status": 401,
    "reasonPhrase": "Unauthorized",
    "body": {
      "fault": {
        "faultstring": "Invalid basic authentication header"
      }
    }
  }
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

19. Encode sets lowercase authorization header on the request

PASS

Policy type: **BasicAuthentication**

Config:

```
{
  "operation": "Encode",
  "user": "myuser",
  "password": "mypass"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {},
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "success",
  "message": "Basic auth header encoded and set on the request (user: bob)",
  "variablesSet": {
    "basicauthentication.header": "Basic Ym9iOmh1bnRlcjI="
  },
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

CORS

20. CORS sets Access-Control-Allow-Origin on response

PASS

Policy type: **CORS**

Config:

```
{
  "allowOrigins": "*",
  "allowMethods": "GET,POST",
  "allowHeaders": "Content-Type"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {},
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "success",
  "message": "CORS headers applied to response",
  "variablesSet": null,
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "GET,POST",
    "Access-Control-Allow-Headers": "Content-Type"
  },
  "body": null
}
```

AccessControl

21. AccessControl DENY matching client IP -> 403

PASS

Policy type: AccessControl

Config:

```
{
  "action": "DENY",
  "ipList": "203.0.113.1"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {},
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "error",
  "message": "Client IP 203.0.113.1 is denied by AccessControl",
  "variablesSet": null,
  "faultResponse": {
    "status": 403,
    "reasonPhrase": "Forbidden",
    "body": {
      "fault": {
        "faultstring": "Access Denied"
      }
    }
  }
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

22. AccessControl DENY non-matching client IP -> success

PASS

Policy type: **AccessControl**

Config:

```
{
  "action": "DENY",
  "ipList": "10.0.0.1"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {},
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "success",
  "message": "Client IP 203.0.113.1 passed AccessControl",
  "variablesSet": null,
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

SpikeArrest

23. 5 rapid calls at rate 3ps -> 4th/5th call blocked (429)

PASS

Policy type: **SpikeArrest**

Config:

```
{
  "rate": "3ps"
}
```

Request (as passed to the executor):

```
{
  "note": "5 sequential calls, same key, within 1 second"
}
```

Result:

```
{
  "status": "error",
  "message": "Spike arrest violation (5/3 in window)",
  "variablesSet": null,
  "faultResponse": {
    "status": 429,
    "reasonPhrase": "Too Many Requests",
    "body": {
      "fault": {
        "faultstring": "Spike arrest violation",
        "detail": {
          "errorCode": "policies.ratelimit.SpikeArrestViolation"
        }
      }
    }
  }
}
```

Quota

24. 4 calls with allowCount=3 -> 4th call blocked (429)

PASS

Policy type: **Quota**

Config:

```
{
  "allowCount": 3,
  "interval": 1,
  "timeUnit": "hour"
}
```

Request (as passed to the executor):

```
{
  "note": "4 sequential calls, same key"
}
```

Result:

```
{
  "status": "error",
  "message": "Quota exceeded (4/3)",
  "variablesSet": null,
  "faultResponse": {
    "status": 429,
    "reasonPhrase": "Too Many Requests",
    "body": {
      "fault": {
        "faultstring": "Quota Violation",
        "detail": {
          "errorCode": "policies.ratelimit.QuotaViolation"
        }
      }
    }
  }
}
```

ResponseCache / PopulateCache / LookupCache

25. PopulateCache then LookupCache round-trips the cached value

PASS

Policy type: **PopulateCache + LookupCache**

Config:

```
{
  "cacheResource": "res1",
  "cacheKey": "k1",
  "ttlSeconds": 60
}
```

Request (as passed to the executor):

```
{
  "populate": {
    "hello": "world"
  },
  "lookupPath": "/x"
}
```

Result:

```
{
  "status": "success",
  "message": "Cache hit",
  "variablesSet": {
    "cached.value": {
      "hello": "world"
    }
  },
  "faultResponse": null
}
```


AssignMessage

26. AssignMessage stores header keys lowercase (X-Client, Authorization)

PASS

Policy type: **AssignMessage**

Config:

```
{
  "assignTo": "request",
  "setHeaders": "X-Client: test-harness\\nAuthorization: Bearer injected-token"
}
```

Request (as passed to the executor):

```
{}
```

Result:

```
{
  "status": "success",
  "message": "Assigned headers/payload to request",
  "variablesSet": null,
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "headers": {
    "x-client": "test-harness",
    "authorization": "Bearer injected-token"
  }
}
```

ExtractVariables

27. Extracts nested JSON path (\$.user.id) into a flow variable

PASS

Policy type: **ExtractVariables**

Config:

```
{
  "source": "request",
  "jsonPath": "$.user.id",
  "variableName": "user.id"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {},
  "query": {},
  "body": {
    "user": {
      "id": 42
    }
  }
}
```

Result:

```
{
  "status": "success",
  "message": "Extracted \"$.user.id\" -> user.id",
  "variablesSet": {
    "user.id": 42
  },
  "faultResponse": null
}
```

RaiseFault

28. Raises the configured status/reason/body

PASS

Policy type: **RaiseFault**

Config:

```
{
  "statusCode": 503,
  "reasonPhrase": "Service Unavailable",
  "errorMessage": "{\"error\":\"maintenance\"}"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {},
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "error",
  "message": "Fault raised: 503 Service Unavailable",
  "variablesSet": null,
  "faultResponse": {
    "status": 503,
    "reasonPhrase": "Service Unavailable",
    "body": {
      "error": "maintenance"
    }
  }
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

JSONTToXML

29. Converts request body from JSON object to XML string

PASS

Policy type: **JSONToXML**

Config:

```
{
  "source": "request"
}
```

Request (as passed to the executor):

```
{
  "body": {
    "a": 1
  }
}
```

Result:

```
{
  "status": "success",
  "message": "Converted request body from JSON to XML",
  "variablesSet": null,
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "body": "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n<root><a>1</a></root>"
}
```

XMLToJSON

30. Converts response body from XML string to a JSON object

PASS

Policy type: **XMLToJSON**

Config:

```
{
  "source": "response"
}
```

Request (as passed to the executor):

```
{
  "body": "<root><a>1</a></root>"
}
```

Result:

```
{
  "status": "success",
  "message": "Converted response body from XML to JSON",
  "variablesSet": null,
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "body": {
    "root": {
      "a": "1"
    }
  }
}
```

JavaScript

31. Sandboxed JS executes and sets a flow variable

PASS

Policy type: JavaScript

Config:

```
{
  "source": "context.setVariable(\"my.var\", 1 + 2);"
}
```

Request (as passed to the executor):

```
{}
```

Result:

```
{
  "status": "success",
  "message": "JavaScript executed",
  "variablesSet": {
    "my.var": 3
  },
  "faultResponse": null
}
```

Route Rules (multi-method target routing)

32. GET /items -> routed to "read-target"

PASS

Policy type: Route Rule evaluation

Config:

```
{
  "routeRules": [
    {
      "name": "reads",
      "targetEndpoint": "read-target",
      "condition": {
        "verb": "GET"
      }
    },
    {
      "name": "writes",
      "targetEndpoint": "write-target"
    }
  ]
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/items"
}
```

Result:

```
{
  "status": "success",
  "message": "resolved target endpoint = \"read-target\"",
  "variablesSet": null,
  "faultResponse": null
}
```

33. POST /items -> routed to "write-target"

PASS

Policy type: Route Rule evaluation

Config:

```
{
  "routeRules": [
    {
      "name": "reads",
      "targetEndpoint": "read-target",
      "condition": {
        "verb": "GET"
      }
    },
    {
      "name": "writes",
      "targetEndpoint": "write-target"
    }
  ]
}
```

Request (as passed to the executor):

```
{
  "method": "POST",
  "path": "/items"
}
```

Result:

```
{
  "status": "success",
  "message": "resolved target endpoint = \"write-target\"",
  "variablesSet": null,
  "faultResponse": null
}
```

34. DELETE /items -> routed to "write-target"

PASS

Policy type: Route Rule evaluation

Config:

```
{
  "routeRules": [
    {
      "name": "reads",
      "targetEndpoint": "read-target",
      "condition": {
        "verb": "GET"
      }
    },
    {
      "name": "writes",
      "targetEndpoint": "write-target"
    }
  ]
}
```

Request (as passed to the executor):

```
{
  "method": "DELETE",
  "path": "/items"
}
```

Result:

```
{
  "status": "success",
  "message": "resolved target endpoint = \"write-target\"",
  "variablesSet": null,
  "faultResponse": null
}
```

35. Verb match is case-insensitive; basePathSuffix requires substring match

PASS

Policy type: **matchesFlowCondition**

Request (as passed to the executor):

```
{}
```

Result:

```
{
  "status": "success",
  "message": "condMatch=true, condNoMatch=true",
  "variablesSet": null,
  "faultResponse": null
}
```

Connector Stubs (simulated)

36. ConcurrentRateLimit executes without throwing

PASS

Policy type: **ConcurrentRateLimit**

Config:

```
{
  "maxConcurrency": 10
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {},
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "success",
  "message": "Concurrency simulated within limit of 10",
  "variablesSet": null,
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

37. FlowCallout executes without throwing

PASS

Policy type: **FlowCallout**

Config:

```
{
  "sharedFlowName": "Common-Security-Flow"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {},
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "success",
  "message": "Shared flow \"Common-Security-Flow\" invoked (see nested steps)",
  "variablesSet": null,
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

38. XSLTransform executes without throwing

PASS

Policy type: **XSLTransform**

Config:

```
{
  "resourceName": "transform.xml",
  "source": "request"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {},
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "success",
  "message": "XSL stylesheet \"transform.xml\" applied to request (simulated passthrough)",
  "variablesSet": null,
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

39. SalesforceConnector executes without throwing

PASS

Policy type: **SalesforceConnector**

Config:

```
{
  "operation": "Create",
  "subject": "Account"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {},
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "success",
  "message": "Salesforce Create on Account simulated",
  "variablesSet": {
    "connector.salesforce.result": {
      "id": "SFd47dpex4",
      "success": true
    }
  },
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```


40. BigQueryConnector executes without throwing

PASS

Policy type: **BigQueryConnector**

Config:

```
{
  "projectId": "my-project"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {},
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "success",
  "message": "BigQuery query executed against my-project (simulated)",
  "variablesSet": {
    "connector.bigquery.rows": [
      {
        "id": 1,
        "value": "sample"
      },
      {
        "id": 2,
        "value": "sample2"
      }
    ]
  },
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

41. CloudStorageConnector executes without throwing

PASS

Policy type: **CloudStorageConnector**

Config:

```
{
  "operation": "Upload",
  "bucket": "my-bucket",
  "objectName": "file.json"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {},
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "success",
  "message": "Cloud Storage Upload on gs://my-bucket/file.json simulated",
  "variablesSet": null,
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```

42. PubSubConnector executes without throwing

PASS

Policy type: **PubSubConnector**

Config:

```
{
  "operation": "Publish",
  "topic": "my-topic"
}
```

Request (as passed to the executor):

```
{
  "method": "GET",
  "path": "/",
  "headers": {},
  "query": {},
  "body": null
}
```

Result:

```
{
  "status": "success",
  "message": "Pub/Sub Publish on topic \"my-topic\" simulated",
  "variablesSet": {
    "connector.pubsub.messageId": "cd8nhf7b1e"
  },
  "faultResponse": null
}
```

Response / context after execution:

```
{
  "status": 200,
  "reasonPhrase": "OK",
  "headers": {},
  "body": null
}
```